

MAJOR PROJECT

On

Campus Management System

Submitted in partial fulfilment of the requirements

for the award of the Degree of

BACHELOR OF COMPUTER APPLICATION

Submitted by

Jay Prakash Kumar (A45304823130)

Shashank Shubham (A45304823141)

Ankit Kumar (A45304823143)

Shubham Singh (A45304823117)

Under the Supervision of

Mr. Suman Kumar Mishra

Assistant Professor, AIIT, AUP

AMITY INSTITUTE OF INFORMATION TECHNOLOGY

AMITY UNIVERSITY

PATNA - 801503

Session 2023-26

Project Certificate

This is to certify that the project report entitled "Smart Campus Management System" submitted to Amity Institute of Information Technology (AIIT), Amity University Patna in partial fulfilment of the requirement for the award of the degree of Bachelor of Computer Application (BCA), is an original work carried out by Mr. Shashank Shubham (A45304823141), Mr. Jay Prakash Kumar (A45304823130), Mr. Ankit Kumar (A45304823143), and Mr. Shubham Singh (A45304823117) under the supervision of Mr. Suman Kumar Mishra.

The matter embodied in this project is genuine work and has not been submitted to this University or to any other University or Institute for the fulfilment of the requirement of any course of study.

Date: _____

Name, Enrollment No. & Signature of the Student

Jay Prakash Kumar (A45304823130) _____

Shashank Shubham (A45304823141) _____

Ankit Kumar (A45304823143) _____

Shubham Singh (A45304823117) _____

Verified by the Project Guide

Mr. Suman Kumar Mishra

Assistant Professor, AIIT

Amity University Patna

Date: _____

Acknowledgement

This project is the outcome of continuous study, practical development effort, faculty guidance, and team coordination during the academic session. We express our sincere gratitude to Amity University Patna for providing the opportunity and academic environment to work on a realistic full-stack software project that connects classroom learning with institutional workflow automation.

We are thankful to the faculty members of Amity Institute of Information Technology for their valuable support in understanding modern web application development, database design, authentication, authorization, REST API design, frontend component structure, and technical documentation. Their guidance helped us convert the project from a simple idea into a working campus portal with separate admin, faculty, and student roles.

We also express special gratitude to our project guide, Mr. Suman Kumar Mishra, for providing direction throughout the development of the Smart Campus Management System. The feedback received during discussions helped refine the authentication workflow, admin-managed account creation, timetable builder, attendance restrictions, student portal design, and report documentation.

Finally, we thank our parents, friends, and classmates for their encouragement and support during the completion of this project. Their motivation helped us continue through design, coding, testing, and documentation stages.

Name, Enrollment No. & Signature of the Student

Jay Prakash Kumar (A45304823130) _____

Shashank Shubham (A45304823141) _____

Ankit Kumar (A45304823143) _____

Shubham Singh (A45304823117) _____

Self-Certificate

This is to certify that the project report entitled Smart Campus Management System is done by Mr. Shashank Shubham (A45304823141), Mr. Jay Prakash Kumar (A45304823130), Mr. Ankit Kumar (A45304823143), and Mr. Shubham Singh (A45304823117). It is authentic work carried out for the partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Application (BCA) under the guidance of Mr. Suman Kumar Mishra.

The matter, analysis, documentation, and software embodied in this project have not been submitted earlier for the award of any degree or diploma to the best of our knowledge and belief.

Name, Enrollment No. & Signature of the Student

Jay Prakash Kumar (A45304823130) _____

Shashank Shubham (A45304823141) _____

Ankit Kumar (A45304823143) _____

Shubham Singh (A45304823117) _____

ABSTRACT

The Smart Campus Management System is a web-based campus portal developed to streamline and digitize important academic and administrative activities within an educational institution. A campus normally handles student records, faculty records, course structures, subject allocation, timetable preparation, lecture attendance, fee records, academic results, notices, messages, and profile information. When these activities are managed through manual registers, paper files, disconnected spreadsheets, and informal communication channels, the institution faces repeated data entry, inconsistent records, delayed access to information, and unnecessary dependency on office staff for routine queries.

This project solves those problems by creating a centralized digital platform where administrators, faculty members, and students interact with one connected system. The application is designed as a role-based portal. The administrator controls institutional data and creates student and faculty accounts. Faculty members view assigned lecture slots and mark attendance only for lectures assigned to them. Students use a portal-style dashboard to view attendance, timetable, fee details, results, messages, notices, and profile information.

The system is implemented using the MERN stack. React.js and Vite are used in the frontend to build responsive pages, reusable panels, dashboard layouts, timetable grids, and student portal sections. Node.js and Express.js are used in the backend to expose REST API routes for authentication, users, courses, subjects, timetable, attendance, payments, results, notices, messages, profiles, and admin statistics. MongoDB is used as the database and Mongoose is used for schema design, validation, indexes, references, and query operations. JSON Web Token authentication protects private routes, bcryptjs secures passwords by hashing them before storage, and Nodemailer supports email-based login ID recovery and password reset OTP delivery.

A significant security improvement in the project is that public registration is disabled. Students log in with roll number, faculty members log in with faculty number, and administrators log in with email. The administrator creates verified accounts directly, which prevents fake student or faculty accounts from being created through open registration. The project also includes OTP-based account recovery through registered email, making the login workflow more realistic and user friendly.

The system includes course and subject management, class assignment, timetable creation, lecture-linked attendance, fee structure and payment records, result entry and summary, role-

specific notices, internal messaging, profile management, and dashboard statistics. The project demonstrates practical use of full-stack development, database modeling, role-based access control, API integration, user interface design, and modular implementation. It can be extended in the future with mobile application support, biometric or QR attendance, push notifications, advanced analytics, cloud deployment, backups, document uploads, and AI-based helpdesk support.

TABLE OF CONTENTS

S. NO	TOPIC	PAGE NO
	Project Certificate	I
	Acknowledgement	II
	Self-Certificate	III
	Abstract	IV
	List of Tables	V
	List of Figures	VI
1	INTRODUCTION 1.1 Introduction 1.2 Problem Statement 1.3 Objectives of the Project 1.4 Aim of the Project 1.5 Scope of the Project 1.6 Structure of the Report	1-8
2	LITERATURE STUDY 2.1 Introduction to Literature Study 2.2 Existing Manual Systems 2.3 Existing ERP and Student Information Systems 2.4 Learning Management Systems and Their Difference 2.5 Limitations of Existing Systems 2.6 Proposed System 2.7 Literature Study Outcome	9-16
3	SYSTEM ANALYSIS 3.1 Requirement Analysis 3.2 Feasibility Analysis 3.3 User Analysis 3.4 Functional Requirements 3.5 Non-Functional Requirements	17-25

	3.6 Data Requirements 3.7 Security Analysis 3.8 Risk Analysis	
4	SYSTEM DESIGN AND ARCHITECTURE 4.1 Overall Architecture 4.2 Frontend Design 4.3 Backend Design 4.4 Database Design 4.5 Authentication and Authorization Design 4.6 API Design 4.7 DFD 4.8 ER Diagram 4.9 Activity and Sequence Flows	26-36
5	SYSTEM REQUIREMENTS 5.1 Hardware Requirements 5.2 Software Requirements 5.3 Programming Platform Features 5.4 Frontend Technology Features 5.5 Backend Technology Features 5.6 Database Technology Features 5.7 Security and Environment Requirements	37-43
6	IMPLEMENTATION AND TESTING 6.1 Implementation Approach 6.2 Authentication Module 6.3 Admin Module 6.4 Academic Module 6.5 Timetable Module 6.6 Attendance Module 6.7 Student Portal Module 6.8 Fees, Results, Notices, Messages	44-56

	6.9 Testing	
7	RESULTS AND SCREENSHOTS 7.1 Results 7.2 Login Page 7.3 Admin Dashboard 7.4 Course and Subject Screens 7.5 Student and Faculty Screens 7.6 Timetable Builder 7.7 Faculty Attendance 7.8 Student Portal 7.9 Advantages and Limitations	57-64
8	CONCLUSION AND FUTURE ENHANCEMENT 8.1 Conclusion 8.2 Limitations 8.3 Future Enhancement 8.4 Final Summary	65-69
	References	70
	Plagiarism Report Copy	71
	Journal Proof / Published Papers / Profile Copy	72

LIST OF TABLES

Table No.	Title	Page
Table 1.1	Problem Areas and Project Responses	Chapter 1
Table 2.1	Comparison Between Manual, ERP, LMS, and Proposed System	Chapter 2
Table 3.1	System Analysis of Users, Inputs, Processes, and Outputs	Chapter 3
Table 3.2	Functional Requirements	Chapter 3
Table 3.3	Non-Functional Requirements	Chapter 3
Table 4.1	System Modules and Responsibilities	Chapter 4
Table 4.2	Database Collections and Major Fields	Chapter 4
Table 4.3	REST API Route Groups	Chapter 4
Table 5.1	Hardware Requirements	Chapter 5
Table 5.2	Software and Technology Requirements	Chapter 5
Table 5.3	Technology Stack Usage Matrix	Chapter 5
Table 6.1	Implementation Module Details	Chapter 6
Table 6.2	Testing Strategy and Expected Results	Chapter 6
Table 7.1	Result Screens and Implemented Features	Chapter 7
Table 8.1	Future Enhancement Plan	Chapter 8

LIST OF FIGURES

Figure No.	Title	Page
Fig 1.1	Manual Campus Workflow Versus Smart Campus Workflow	Chapter 1
Fig 3.1	Level 0 DFD Context Diagram	Chapter 3
Fig 3.2	Level 1 DFD Detailed Flow	Chapter 3
Fig 4.1	System Architecture Diagram	Chapter 4
Fig 4.2	Entity Relationship Diagram	Chapter 4
Fig 4.3	Authentication Sequence Flow	Chapter 4
Fig 5.1	Development Environment Layout	Chapter 5
Fig 5.2	Technology Stack Layer Diagram	Chapter 5
Fig 6.1	Implementation Flow by Module	Chapter 6
Fig 6.2	Testing Workflow	Chapter 6
Fig 7.1	Login Page	Chapter 7
Fig 7.2	Admin Dashboard	Chapter 7
Fig 7.3	Course and Subject Management	Chapter 7
Fig 7.4	Student Management Screen	Chapter 7
Fig 7.5	Timetable Builder	Chapter 7
Fig 7.6	Faculty Attendance Screen	Chapter 7
Fig 7.7	Student Portal Dashboard	Chapter 7
Fig 7.8	Fee, Result, Notice, and Message Screens	Chapter 7

CHAPTER 1

INTRODUCTION

1.1 Introduction

Educational institutions handle a large volume of academic and administrative information every day. Student admission details, roll numbers, course structures, subject lists, class batches, faculty assignments, timetables, attendance records, fee dues, academic marks, notices, and communication records must remain available, accurate, secure, and up to date. In many institutions these activities are still managed through paper registers, office files, separate spreadsheet sheets, message groups, and manual checking by administrative staff.

The Smart Campus Management System is developed to convert these scattered activities into a centralized web-based application. Instead of requiring students to visit different offices for information, the system gives them a single portal. Instead of requiring faculty members to keep separate attendance registers and later submit attendance manually, the system gives faculty members a lecture-wise attendance interface. Instead of allowing public registration, which can create fake accounts, the system gives account creation authority to the administrator.

The website in the project folder is a full-stack implementation. The frontend is built with React.js, Vite, React Router, Axios, and CSS. The backend is built with Node.js, Express.js, MongoDB, Mongoose, JWT, bcryptjs, Nodemailer, dotenv, and cors. This means the project does not only show static pages; it implements actual API communication, protected routes, password hashing, database schemas, REST endpoints, and role-specific screens.

```
Manual workflow: Student/Faculty/Admin -> Paper register/spreadsheet -> Office  
processing -> Delayed information  
  
Smart Campus workflow: User -> React Portal -> Express API -> MongoDB -> Real-time  
dashboard response  
  
Improvement: one source of data, role-based access, faster retrieval, lower  
duplication, safer account control
```

Fig 1.1: Manual Campus Workflow Versus Smart Campus Workflow

1.2 Problem Statement

The main problem addressed by this project is the absence of a centralized and secure digital platform for managing routine campus operations. When attendance, timetable, student records, fee

status, result details, and notices are maintained separately, users cannot access information quickly and administrators cannot maintain a reliable single version of institutional data.

Manual attendance creates several difficulties. A faculty member may mark attendance in a paper register, then later submit it to the office, and then an admin may enter it into another record. This increases the chance of mistakes, missing entries, duplicate records, and delay in student visibility. The implemented project improves this by linking attendance to timetable entries. A faculty member can load only assigned lectures for a date and save present/absent records for students in that class batch.

Open registration is another important problem. In a real campus portal, every student and faculty member must be an authorized institutional user. If anyone can register freely, a person outside the university could create a student or faculty account. This project disables public registration in the backend and returns a clear response that users must contact the administrator. The admin creates student accounts with roll numbers and faculty accounts with faculty numbers, and those institutional identifiers become login IDs.

Problem Area	Effect in Manual/Weak System	Project Response
Student records	Records are spread across registers and spreadsheets.	Admin creates and edits student profile records through StudentsPanel and /api/users/students routes.
Faculty records	Faculty information and assignment details are not connected with timetable.	Faculty accounts and profiles are created by admin and connected with timetable and class assignments.
Attendance	Paper attendance requires later entry and may not match actual lecture slots.	AttendanceForm loads only assigned faculty lectures and stores lecture-wise attendance.
Timetable	Manual timetable preparation can create faculty, class, or room clashes.	Timetable routes include conflict detection for faculty, class, and room overlap.
Fees	Students need office confirmation for dues and receipts.	Payment records and fee structures are available through payment

		APIs and portal screens.
Results	Marks are often shared separately and not visible in one portal.	ResultsPanel stores result records and computes marksheet-style summaries.
Communication	Notices and messages may be scattered across informal channels.	Notice and message modules provide role-targeted announcements and threaded communication.
Security	Open registration can create unauthorized accounts.	Public registration is disabled; admin creates verified users and JWT protects API access.

1.3 Objectives of the Project

- To develop a centralized web-based campus management portal that can be accessed by administrators, faculty members, and students through a browser.
- To provide secure role-based authentication for admin, faculty, and student users.
- To disable public registration and allow only admin-created student and faculty accounts.
- To allow students to log in using roll number, faculty members to log in using faculty number, and administrators to log in using email.
- To implement password security using bcryptjs hashing instead of storing plain text passwords.
- To implement JWT-based authorization so protected API requests include a verified bearer token.
- To provide forgot ID and forgot password functionality through registered email and OTP recovery.
- To allow admin users to manage courses, semester/year duration, departments, and related academic structure.
- To allow admin users to manage subjects by course, semester/year, subject code, subject type, theory marks, and practical marks.
- To provide student and faculty account management from the admin dashboard.
- To provide class assignment and roster management so students and faculty can be connected with class batches.
- To create a visual timetable builder where the admin can assign subject, faculty, room, day, and period.
- To restrict faculty attendance marking to assigned lecture slots only.
- To provide a student portal with attendance summary, timetable, fee records, result records, messages, and profile information.
- To support notices targeted to students, faculty, or all users.
- To provide internal messaging between faculty and students, with conversation threading.
- To store all persistent data in MongoDB through Mongoose schemas.

- To create a modular codebase where frontend panels connect to backend route groups cleanly.

1.4 Aim of the Project

The aim of the Smart Campus Management System is to design and develop a secure, user-friendly, role-based, and database-driven campus portal for managing important academic and administrative activities of an institution. The system aims to reduce manual workload, improve data accuracy, protect institutional access, and provide faster access to information for administrators, faculty members, and students.

The project also aims to demonstrate practical understanding of full-stack development. The frontend shows how React components can create dashboards and interactive panels. The backend shows how Express routes and Mongoose models can implement business rules. The database shows how campus records can be structured into collections. The authentication system shows how JWT, password hashing, and OTP-based recovery can make the application more secure and realistic.

1.5 Scope of the Project

The scope of this project includes the development of a working web application for campus management. It covers administrator operations, faculty operations, and student operations. The administrator can manage institutional records such as courses, subjects, students, faculty, class assignments, timetable, notices, payments, results, and dashboard statistics. Faculty members can access assigned timetable entries, mark lecture-wise attendance, enter academic results, and communicate with students. Students can use a portal-style dashboard to view their academic data and personal profile information.

The system does not claim to replace every enterprise-level feature of a commercial campus ERP. It focuses on the most important academic project modules and implements them as a functional MERN stack website. Features such as biometric attendance, cloud file storage, push notification service, production backup automation, mobile app packaging, and advanced analytics are listed as future enhancements.

1.6 Structure of the Report

- Chapter 1 introduces the project, explains the problem statement, lists objectives, defines the aim, and describes the scope.
- Chapter 2 presents the literature study by discussing manual systems, ERP systems, student information systems, learning management systems, their limitations, and the proposed system.
- Chapter 3 explains system analysis, including requirement analysis, feasibility, user analysis, functional requirements, non-functional requirements, data needs, security analysis, and risk analysis.
- Chapter 4 describes system design and architecture, including frontend design, backend design, database design, authentication design, API route design, DFD, ER diagram, activity flow, and sequence flow.
- Chapter 5 explains hardware requirements, software requirements, programming platform features, and exactly where each technology stack component is used.
- Chapter 6 explains implementation by module and describes testing without including source code.
- Chapter 7 presents results and screenshots documentation for the implemented website screens.

- Chapter 8 concludes the project and explains future enhancements.

CHAPTER 2

LITERATURE STUDY

2.1 Introduction to Literature Study

A literature study for a campus management system examines how educational institutions traditionally manage campus data and how modern software systems improve these workflows. The purpose is not only to mention existing systems but also to understand their strengths, limitations, and relevance to the proposed Smart Campus Management System.

Campus administration involves both academic and non-academic work. Academic work includes course structure, subjects, timetable, attendance, marks, results, and faculty assignments. Administrative work includes user records, profiles, fees, notices, and communication. Existing tools may solve one part of the problem but may not provide a complete, secure, role-based, integrated workflow suitable for a small academic full-stack project.

2.2 Existing Manual Systems

Manual campus management systems rely on paper registers, attendance sheets, office files, printed timetables, notice boards, and verbal communication. These methods are familiar and inexpensive at the beginning, but they become inefficient as the number of students, faculty, subjects, and classes increases. A paper register can record attendance, but it cannot instantly calculate subject-wise attendance percentage for students. A notice board can display announcements, but it cannot ensure that students and faculty access targeted notices remotely.

Manual systems are also difficult to audit. If an attendance entry is overwritten, if a fee receipt is misplaced, or if a timetable is modified informally, it becomes hard to identify the latest accurate record. The Smart Campus Management System addresses this by storing records in MongoDB and exposing them through modules that can be accessed according to user role.

2.3 Existing ERP and Student Information Systems

Enterprise Resource Planning systems and Student Information Systems are widely used in universities. They usually provide modules for student admission, fee management, timetable, attendance, examination, payroll, HR, hostel, transport, library, and communication. Such systems are powerful but often large, expensive, and difficult to customize for a focused academic project.

The proposed project adopts the useful idea of centralization from ERP systems but keeps the module set understandable and implementable. Instead of trying to build every enterprise feature, it focuses on a realistic subset: authentication, users, profiles, courses, subjects, timetable, attendance, payments, results, notices, messages, and statistics.

2.4 Learning Management Systems and Their Difference

Learning Management Systems such as classroom platforms and online course systems focus mainly on teaching content, assignments, course material, quizzes, discussion, and online learning delivery. They are very useful for teaching and learning but they are not always designed to manage institutional administration such as fee records, account creation by admin, faculty number login, roll number login, class roster assignment, or lecture-linked attendance.

The Smart Campus Management System is closer to a campus portal than a pure learning management system. It does not attempt to deliver online course content. Instead, it manages operational data required by students, faculty, and administrators.

System Type	Main Focus	Strength	Limitation	Relevance to Proposed System
Manual registers	Paper-based records	Simple and familiar	Slow, duplicate, difficult to search	Shows why digitization is needed
Spreadsheets	Tabular data entry	Easy calculations and filtering	Disconnected and weak access control	Useful idea of structured data but not enough for role-based portal
ERP systems	Full institutional automation	Large integrated feature set	Expensive and complex	Provides concept of central campus database
Student information systems	Student records and academics	Strong student profile and academic data handling	May not cover messaging or custom workflows	Inspires profile, result, fee, and attendance modules
Learning management systems	Course delivery	Good for assignments and course material	Different from administrative campus management	Clarifies why this project is a campus portal
Proposed system	Role-based campus operations	Focused, customizable, full-	Can be expanded further	Implements core campus workflows

		stack project		using MERN stack
--	--	---------------	--	------------------

2.5 Limitations of Existing Systems

- Manual and semi-digital systems require repeated data entry, which increases errors and wastes time.
- Disconnected tools prevent the institution from maintaining one accurate source of information.
- Open registration creates the risk of unauthorized users creating fake student or faculty accounts.
- Faculty attendance may not be connected to actual assigned lecture slots.
- Timetable creation can create conflicts when class, faculty, and room availability are checked manually.
- Students may need to depend on office staff for information that should be visible through a portal.
- Commercial ERP systems can be too expensive, complex, and difficult to customize for a student project.
- Simple websites may show static pages but do not implement database-backed workflows.

2.6 Proposed System

The proposed system is a MERN stack campus portal that integrates core academic and administrative modules into one role-based website. The administrator becomes the central controller of institutional records. The admin creates verified students and faculty, manages course and subject data, builds timetables, creates notices, manages payment records, enters or supervises results, and views dashboard statistics.

Faculty users receive a separate experience. They do not get admin controls. Their timetable view is filtered to assigned entries, and the attendance module loads only lecture slots assigned to the logged-in faculty member. This makes attendance realistic because a faculty member cannot mark an arbitrary subject or class without timetable assignment.

Student users receive a portal-style dashboard instead of the same admin panel. The student portal has sidebar navigation and sections for home, classes, attendance, fee details, results, messages, and profile. This makes the student experience closer to real academic portals where students view personal academic information rather than manage institutional data.

2.7 Literature Study Outcome

The outcome of the literature study is that a successful campus management system should be centralized, role-based, secure, modular, and easy to use. It should separate administrator functions from faculty and student functions. It should keep authentication controlled and should connect attendance with timetable assignments. It should store data in a persistent database and should expose clear APIs for frontend interaction.

The implemented project follows these findings. MongoDB and Mongoose provide structured persistence, Express provides REST API modules, React provides role-specific screens, JWT secures protected access, bcryptjs secures passwords, Nodemailer supports recovery workflows, and the dashboard modules represent practical campus operations.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Requirement Analysis

System analysis identifies what the system must do, who will use it, what data it must store, what restrictions must apply, and what quality expectations must be satisfied. For the Smart Campus Management System, the analysis begins with three roles: administrator, faculty, and student. Each role has different responsibilities and therefore needs a different user interface and API permission level.

The administrator needs full control over institutional records. The faculty member needs controlled access to assigned academic work. The student needs read access to personal academic information. These role differences are reflected in the frontend route guard, dashboard rendering logic, backend middleware, and route-level permission checks.

3.2 Feasibility Analysis

Technical feasibility is strong because the selected technologies are widely used for web application development. React and Vite provide a modern frontend environment. Node.js and Express provide a lightweight backend server. MongoDB and Mongoose support flexible document data, references, indexes, and schema validation. JWT and bcryptjs are common choices for authentication and password security.

Operational feasibility is also strong because the system follows familiar campus workflows. Students already understand roll numbers, faculty members already understand faculty IDs and lecture periods, and administrators already manage courses, subjects, timetables, fees, and results. The project digitizes these familiar workflows instead of forcing a completely new administrative model.

Economic feasibility is suitable for an academic project because the stack can run locally without paid infrastructure. MongoDB can run locally, the backend uses open-source packages, and the frontend can be developed with Vite. Production deployment may require hosting, SSL, backups, and SMTP service, but the development version is low cost.

3.3 User Analysis

User	Primary Needs	Allowed Actions	Restricted Actions
Administrator	Full campus management and monitoring	Create users, courses, subjects, timetable, notices, payments, results, view stats, manage records	Should not bypass validation or expose passwords
Faculty	Academic work for assigned lectures and students	View assigned timetable, mark assigned attendance, enter results, message students, view notices	Cannot create users, courses, or arbitrary timetable slots
Student	Personal academic information and communication	View timetable, attendance, fees, results, messages, profile, notices	Cannot modify institutional records or mark attendance

3.4 Functional Requirements

Req. ID	Requirement	Implementation Evidence
FR-01	The system shall allow login using student roll number, faculty number, or admin email.	Implemented through authController findByIdentifier and Login.jsx identifier field.
FR-02	The system shall disable public registration.	Implemented by /api/auth/register returning status 410.
FR-03	The system shall generate JWT tokens after valid login.	Implemented in authController login function.
FR-04	The system shall protect private APIs using middleware.	Implemented in authMiddleware.js and used in route groups.
FR-05	The admin shall create student accounts.	Implemented by /api/users/students and StudentsPanel.jsx.
FR-06	The admin shall create faculty accounts.	Implemented by /api/users/faculty and FacultiesPanel.jsx.
FR-07	The admin shall manage courses.	Implemented by /api/courses and CoursesPanel.jsx.
FR-08	The admin shall manage subjects by course and semester/year.	Implemented by /api/subjects and CourseSubjectsPanel.jsx.
FR-09	The admin shall build timetable slots.	Implemented by /api/timetable/slot and TimetablePanel.jsx.
FR-10	The system shall detect timetable conflicts.	Implemented in timetableRoutes findConflict logic.
FR-11	Faculty shall mark attendance only for assigned lectures.	Implemented in attendanceRoutes getFacultyLecture validation.
FR-12	Students shall view attendance summary.	Implemented by /api/attendance/summary/:studentId and StudentPortal.jsx.
FR-13	The system shall manage fee structures and payment records.	Implemented by payment routes and PaymentsPanel.jsx.
FR-14	The system shall manage results	Implemented by result routes and

	and summary data.	ResultsPanel.jsx.
FR-15	The system shall support notices.	Implemented by notice routes and NoticeForm/Dashboard.
FR-16	The system shall support student-faculty messaging.	Implemented by message routes and MessagesPanel.jsx.
FR-17	The system shall support profile view and update.	Implemented by profile routes and ProfilePanel.jsx.
FR-18	The admin dashboard shall show institutional counts.	Implemented by /api/admin/stats and Dashboard.jsx stats cards.

3.5 Non-Functional Requirements

Quality Area	Requirement	How Project Supports It
Security	Passwords must not be stored as plain text.	bcryptjs hashes passwords during admin-created account creation and password reset.
Authorization	Users must access only role-appropriate features.	JWT middleware and route-level role checks restrict protected operations.
Usability	Screens should be clear and role-specific.	Dashboard renders admin/faculty modules and StudentPortal renders student navigation.
Maintainability	Modules should be separated.	Backend route files, models, controllers, middleware, and frontend components are separated.
Scalability	Database structure should allow more modules later.	Mongoose models can be extended with new fields and collections.
Reliability	Duplicate attendance for the same student and lecture should be prevented.	Attendance schema includes a unique compound index.
Performance	Dashboard counts should load	Admin stats route uses Promise.all

	together.	for multiple counts.
Recoverability	Users should recover login ID and reset passwords.	Nodemailer and OTP hash fields support recovery workflows.

3.6 Data Requirements

The system requires identity data, academic data, scheduling data, attendance data, payment data, result data, communication data, and profile data. Identity data is stored in the User collection and includes name, email, institutional ID, password hash, role, status, and recovery fields. Academic data is stored across Course, Subject, ClassAssignment, Timetable, and Result collections. Operational data is stored in Attendance, Payment, FeeStructure, Notice, Message, and Profile collections.

Collection	Major Data Stored	Reason for Requirement
User	name, email, institutionalId, password, emailVerified, OTP hashes, role, status	Stores login identity, role, approval status, hashed password, and account recovery state for admin, faculty, and student users.
Profile	user, course, semester, department, rollNumber, facultyNumber, phone, address, personal/family details, assignedClass, assignedBatch, assignedSubjects	Stores academic and personal profile information separate from authentication details so role-specific records can grow without changing the login schema.
Course	courseCode, courseName, semYearType, totalSemYear, department	Defines academic programs and whether a program follows semesters or years.
Subject	course, courseCode, subjectCode, name, semester, subjectType, theoryMarks, practicalMarks	Connects subjects to course and semester/year combinations and records whether a subject is core or specialisation elective.
ClassAssignment	className, batch, course, semester, department, classTeacher, subjects	Maps class batches to course structure, student rosters, and faculty subject assignments.
Timetable	course, semester, className, batch, classTeacher, subject, faculty, day, startTime, endTime, room	Stores lecture slots used by the timetable board and the attendance validation process.
Attendance	studentId, faculty, timetableEntry,	Stores lecture-wise attendance

	subject, className, batch, lectureDate, day, startTime, endTime, status	records with a unique index to prevent duplicate entries for the same student and lecture.
FeeStructure	className, semester, feeType, amount, dueDate, notes	Defines reusable fee templates that can be assigned to a class and semester.
Payment	student, className, semester, feeType, amount, dueDate, feeStructure, status, transactionId, receiptNumber, paidAt	Stores individual student fee obligations, payment status, and receipt metadata.
Result	student, subject, course, semester, marksObtained, maxMarks, grade, remarks, recordedBy	Stores marks and grade entries used to calculate semester summaries, SGPA-style snapshots, and CGPA-style summaries.
Notice	title, content, role, createdAt	Stores admin-created announcements targeted to all users or selected roles.
Message	sender, receiver, conversationId, subject, body, replyTo, readAt	Stores threaded internal messages between students and faculty, with admin read visibility.

3.7 Security Analysis

Security is important because the application stores student identity, academic records, fees, marks, messages, and profile information. The first security decision is to disable public registration. This means a student or faculty account cannot be created by an unknown visitor. Only an administrator can create approved accounts for students and faculty. The backend register endpoint intentionally responds with a disabled registration message.

The second security decision is password hashing. When admin creates a student or faculty account, the backend hashes the password using bcryptjs. When the user logs in, bcryptjs compares the entered password with the stored hash. This prevents plain text passwords from being stored in MongoDB.

The third security decision is token-based authentication. After successful login, the backend signs a JWT containing the user ID and role. The frontend stores that token in localStorage and sends it in the Authorization header through the Axios interceptor. The backend authMiddleware verifies the token before allowing access to protected routes.

3.8 Risk Analysis

Risk	Impact	Control in Current Project	Future Improvement
SMTP not configured	OTP emails cannot be delivered in production.	emailService returns a development fallback reason when SMTP is missing.	Configure secure SMTP provider and remove development OTP exposure.
MongoDB server unavailable	Application cannot store or retrieve data.	Backend uses MONGO_URI and logs connection errors.	Add health checks, backups, and retry/monitoring.
Weak initial passwords	Accounts may be guessed.	Backend checks minimum password length for admin-created users.	Add stronger password policy and forced first login password change.
LocalStorage token exposure	Token could be read by malicious browser scripts.	JWT expiry limits token lifetime.	Use httpOnly secure cookies in production.
No file storage system	Photo URL is basic and not a managed upload.	Profile supports photoUrl field.	Add cloud storage with validation and malware scanning.
No production deployment setup	System is development-focused.	Local development scripts are available.	Add Docker, HTTPS, CI/CD, logs, and backups.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall Architecture

The Smart Campus Management System follows a client-server architecture. The frontend client is a React application running in the browser. The backend server is an Express application running on Node.js. The database is MongoDB accessed through Mongoose models. Communication between frontend and backend happens through REST API calls using Axios.

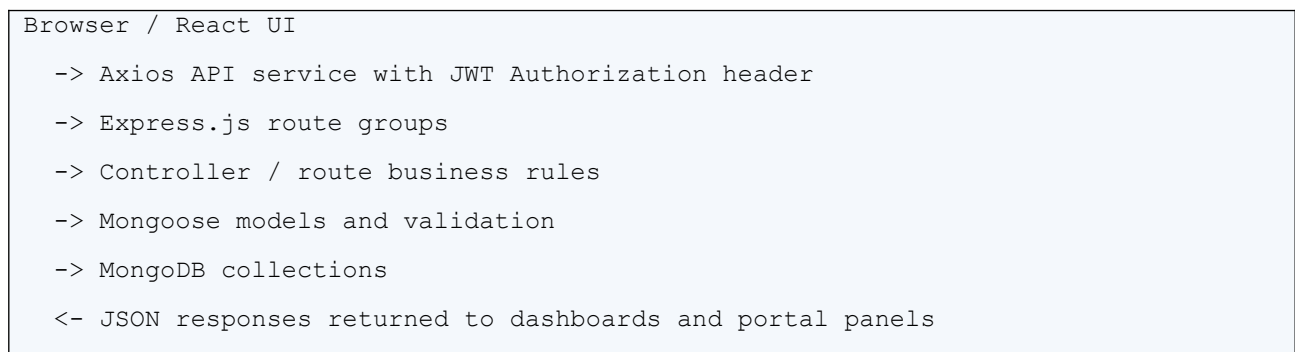


Fig 4.1: System Architecture Diagram

4.2 Frontend Design

The frontend is organized around pages, reusable components, styling files, and an API service. App.jsx defines routes for login, dashboard, and profile. ProtectedRoute.jsx checks token and role before rendering protected screens. Login.jsx handles login, forgot ID, and forgot password modes. Dashboard.jsx reads the logged-in user and displays admin, faculty, or student experience accordingly.

The admin interface is composed of panels. CoursesPanel manages course records. CourseSubjectsPanel manages course-linked subjects. StudentsPanel creates and edits students. FacultiesPanel creates faculty accounts. TimetablePanel builds the schedule grid. PaymentsPanel manages fees and receipts. ResultsPanel records and displays academic marks. MessagesPanel handles threaded communication. NoticeForm creates notices.

The student interface is handled by StudentPortal.jsx. It uses a sidebar navigation model with sections for Home, My Classes, My Attendance, Fee Details, Results, Messages, and Profile. It

loads attendance, summary, payments, results, timetable, messages, and profile data through Promise.all, then presents the information through cards, boards, rings, rows, and portal panels.

Frontend File	Purpose	Technology Usage
Login.jsx	Login, forgot ID, and forgot password screens	Uses React state for form modes; calls auth endpoints with Axios; stores token and user object in localStorage after successful login.
Dashboard.jsx	Role-based dashboard router	Reads localStorage user, displays admin/faculty dashboards, delegates student users to StudentPortal, loads notices and admin statistics.
ProtectedRoute.jsx	Client-side route guard	Checks localStorage token and role before allowing access to dashboard and profile routes.
CoursesPanel.jsx	Course management	Lists courses, creates course records, and displays counts for related subjects and students.
CourseSubjectsPanel.jsx	Subject management	Loads courses, filters by selected course and semester/year, creates subjects with code, type, theory marks, and practical marks.
StudentsPanel.jsx	Student management	Admin creates and edits student accounts, filters by course/semester/enrollment, and stores roll number as institutional login ID.
FacultiesPanel.jsx	Faculty management	Admin creates faculty accounts with faculty number, profile fields, qualification, experience, and initial password.
ClassAssignmentPanel.jsx	Class and roster management	Maintains class/batch records, subjects, student rosters, faculty

		assignments, and class teacher selection.
TimetablePanel.jsx	Visual timetable builder	Renders Monday-Friday schedule grid, lets admin choose slots, assigns subject/faculty/room, and provides read-only timetable board for faculty/student roles.
AttendanceForm.jsx	Faculty attendance entry	Loads only assigned lectures for selected date, retrieves class students, defaults attendance, and saves present/absent records.
StudentPortal.jsx	Student portal	Provides sidebar sections for home, classes, attendance, fees, results, messages, and profile using combined API data.
PaymentsPanel.jsx	Fees and receipts	Admin creates fee structures/manual records, assigns class fees, students/admin mark paid, and receipts are shown after payment.
ResultsPanel.jsx	Result entry and marksheet view	Admin/faculty create and edit marks; students view results, percentage, SGPA-style and CGPA-style summary data.
MessagesPanel.jsx	Internal messaging	Students and faculty start or continue threads; admin can view conversations; unread counts are updated.
NoticeForm.jsx	Notice creation	Admin creates notices targeted to student, faculty, or all roles.

4.3 Backend Design

The backend is built with Express.js. `server.js` imports route groups, enables CORS for the frontend origin, parses JSON request bodies, defines a health endpoint, mounts API route groups, connects to MongoDB through Mongoose, and starts the server. The route groups are separated by domain so that authentication, users, attendance, courses, subjects, timetable, payments, results, notices, messages, class assignments, profiles, and admin statistics remain understandable.

Route Group	Endpoints	Responsibility
Auth	<code>/api/auth/login</code> , <code>/recover-id</code> , <code>/request-password-reset</code> , <code>/reset-password</code> , <code>/register</code>	Performs login with roll number, faculty number, or admin email; disables public registration; sends login ID and OTP recovery emails through Nodemailer.
Admin Stats	<code>/api/admin/stats</code>	Counts students, faculty, courses, subjects, notices, attendance records, payments, timetable entries, and results for the admin dashboard cards.
Users	<code>/api/users/students</code> , <code>/students/manage</code> , <code>/students/:profileId</code> , <code>/faculty</code> , <code>/faculty/manage</code> , <code>/approved</code>	Creates and lists students/faculty; allows admin editing of student details; exposes approved users for profiles and module selection.
Profiles	<code>/api/profiles</code> , <code>/me</code> , <code>/user/:userId</code>	Provides profile view and update operations for self-service and admin-controlled profile management.
Courses	<code>/api/courses</code>	Creates, lists, updates, and deletes course records with course code, name, department, duration type, and total semesters/years.
Subjects	<code>/api/subjects</code>	Creates and lists course-linked subjects by course and semester/year with subject code,

		type, and marks division.
Class Assignments	/api/class-assignments	Creates class batches, rosters, and faculty-subject mappings and validates class membership for attendance.
Timetable	/api/timetable, /slot, /:id	Creates and updates visual timetable slots while detecting conflicts for faculty, class, and room overlaps.
Attendance	/api/attendance/faculty/lectures, /faculty/session, /summary/:studentId, /:studentId	Restricts faculty attendance to assigned lecture slots and exposes student attendance records and subject-wise percentages.
Payments	/api/payments, /structures, /structures/:id/assign, /:id/pay, /:id/receipt	Manages fee structures, assigns them to students, marks payments, and returns receipts.
Results	/api/results, /summary	Records marks, updates results, and builds student summary data for marksheet-style presentation.
Notices	/api/notices	Lets admins create, edit, and delete notices while students and faculty retrieve role-relevant announcements.
Messages	/api/messages, /threads, /threads/:conversationId, /read	Provides student-faculty conversations, unread counts, thread reads, and admin oversight.

4.4 Database Design

The database design uses separate MongoDB collections for separate campus concepts. The User model stores identity and authentication state. The Profile model stores extended personal and academic data. Course and Subject store academic structure. ClassAssignment connects class batches, rosters, and faculty subjects. Timetable stores schedule slots. Attendance stores lecture-

wise attendance. Payment and FeeStructure store fee data. Result stores academic marks. Notice stores announcements. Message stores conversation threads.

Entity	Key Fields	Explanation
User	name, email, institutionalId, password, emailVerified, OTP hashes, role, status	Stores login identity, role, approval status, hashed password, and account recovery state for admin, faculty, and student users.
Profile	user, course, semester, department, rollNumber, facultyNumber, phone, address, personal/family details, assignedClass, assignedBatch, assignedSubjects	Stores academic and personal profile information separate from authentication details so role-specific records can grow without changing the login schema.
Course	courseCode, courseName, semYearType, totalSemYear, department	Defines academic programs and whether a program follows semesters or years.
Subject	course, courseCode, subjectCode, name, semester, subjectType, theoryMarks, practicalMarks	Connects subjects to course and semester/year combinations and records whether a subject is core or specialisation elective.
ClassAssignment	className, batch, course, semester, department, classTeacher, subjects	Maps class batches to course structure, student rosters, and faculty subject assignments.
Timetable	course, semester, className, batch, classTeacher, subject, faculty, day, startTime, endTime, room	Stores lecture slots used by the timetable board and the attendance validation process.
Attendance	studentId, faculty, timetableEntry, subject, className, batch, lectureDate, day, startTime, endTime, status	Stores lecture-wise attendance records with a unique index to prevent duplicate entries for the same student and lecture.
FeeStructure	className, semester, feeType, amount, dueDate, notes	Defines reusable fee templates that can be assigned to a class and semester.

Payment	student, className, semester, feeType, amount, dueDate, feeStructure, status, transactionId, receiptNumber, paidAt	Stores individual student fee obligations, payment status, and receipt metadata.
Result	student, subject, course, semester, marksObtained, maxMarks, grade, remarks, recordedBy	Stores marks and grade entries used to calculate semester summaries, SGPA-style snapshots, and CGPA-style summaries.
Notice	title, content, role, createdAt	Stores admin-created announcements targeted to all users or selected roles.
Message	sender, receiver, conversationId, subject, body, replyTo, readAt	Stores threaded internal messages between students and faculty, with admin read visibility.

4.5 Entity Relationship Diagram

```
User 1 ---- 1 Profile
Course 1 ---- many Subject
Course + Semester ---- many Timetable entries
ClassAssignment 1 ---- many Profile student roster records
Faculty User 1 ---- many Timetable entries
Timetable 1 ---- many Attendance records
Student User 1 ---- many Attendance records
Student User 1 ---- many Payment records
Student User 1 ---- many Result records
User 1 ---- many sent Message records
User 1 ---- many received Message records
Admin User ---- many Notice records logically through dashboard action
```

Fig 4.2: Entity Relationship Diagram

4.6 Authentication and Authorization Design

Authentication begins when the user enters an identifier and password. The identifier may be a roll number, faculty number, or email. The backend normalizes the input and searches by institutionalId or email. If the user exists, bcryptjs compares the entered password with the stored hash. If the account is active and emailVerified is true, the backend signs a JWT containing user ID and role. The frontend stores the token and user object in localStorage.

Authorization happens in two places. On the frontend, ProtectedRoute checks whether a token exists and whether the role is allowed for the route. On the backend, authMiddleware verifies the JWT and each sensitive route checks req.user.role before performing admin-only, faculty-only, or student-limited operations.

```
User submits identifier + password
Frontend sends POST /api/auth/login
Backend finds user by institutionalId or email
bcryptjs compares password with stored hash
Backend checks account active state
jsonwebtoken signs token with id and role
Frontend stores token and user object
Axios interceptor attaches Bearer token to protected requests
authMiddleware verifies token and route checks role
```

Fig 4.3: Authentication Sequence Flow

4.7 Data Flow Diagram

```
[Admin] ---- manages users/courses/subjects/timetable/fees/results/notices ---->
                                [Smart Campus Management System] <---- stores/retrieves ----
[MongoDB]
[Faculty] -- views assigned lectures, marks attendance, records results, messages
-->
[Student] -- views attendance/timetable/fees/results/messages/profile/notices
----->
```

Fig 3.1: Level 0 DFD Context Diagram

```
1. Login Process: User -> Auth API -> User collection -> JWT response
2. Admin Management: Admin -> User/Course/Subject/Profile APIs -> MongoDB
3. Timetable Process: Admin -> Timetable API -> conflict check -> Timetable
collection
4. Attendance Process: Faculty -> Attendance API -> Timetable validation ->
Attendance collection
5. Student Portal: Student -> APIs ->
Attendance/Payments/Results/Timetable/Profile collections
6. Communication: Student/Faculty -> Message API -> Message collection -> thread
view
7. Notice Process: Admin -> Notice API -> Notice collection -> role-filtered
display
```

Fig 3.2: Level 1 DFD Detailed Flow

4.8 Activity Diagram / Flow Chart

```
Start
Open Smart Campus portal
Enter login identifier and password
System verifies user and role
If admin -> open admin dashboard -> manage institutional records
If faculty -> open faculty dashboard -> view timetable and mark attendance
If student -> open student portal -> view academic information
User performs allowed action
System reads/writes MongoDB through Express API
User logs out
End
```

Fig 4.4: Activity Diagram

4.9 API Design Principles

The API design is grouped by resource names and uses common HTTP methods. GET retrieves data, POST creates records or starts actions, PUT updates records or saves existing slots, and DELETE removes records. Responses are JSON objects or arrays. Protected routes require JWT authentication, and role checks are placed before actions that change institutional data.

CHAPTER 5

SYSTEM REQUIREMENTS

5.1 Hardware Requirements

Hardware Component	Minimum Requirement	Reason
Processor	Intel i3 / Ryzen 3 or equivalent	Enough for running browser, Node.js backend, frontend dev server, and MongoDB locally.
RAM	4 GB minimum, 8 GB recommended	Development tools, browser tabs, and database processes work more smoothly with higher memory.
Storage	500 GB HDD/SSD recommended	Required for project files, node_modules, MongoDB data, screenshots, and documentation.
Display	1024 x 768 minimum, 1366 x 768 or higher recommended	Dashboard tables, timetable grid, and portal panels need adequate screen width.
Internet	Required for email OTP, package installation, documentation references, and deployment	Nodemailer SMTP and dependency installation need network connectivity.

5.2 Software Requirements

Category	Technology	Usage in Project
Operating System	Windows / Linux / macOS	The project can run on common operating systems that support Node.js and MongoDB.
Frontend Runtime	Browser with JavaScript support	React application runs in the browser.
Code Editor	Visual Studio Code	Used for editing frontend, backend, and documentation files.

Frontend Build Tool	Vite	Runs development server and creates production build.
Backend Runtime	Node.js	Runs the Express server.
Package Manager	npm	Installs frontend and backend dependencies.
Database	MongoDB	Stores application records.
API Testing	Browser / REST client	Can be used to verify API endpoints.
Version Control	Git	Maintains project history and collaboration changes.

5.3 Programming Platform Features

The programming platform is divided into frontend, backend, and database layers. The frontend uses component-based development, state hooks, effect hooks, routing, Axios API calls, and CSS layout. The backend uses route handlers, middleware, schema models, validation, asynchronous database queries, and JSON responses. The database layer uses flexible documents with references and indexes.

campus/
frontend/ -> React + Vite application
backend/ -> Node.js + Express API
backend/models/ -> Mongoose schemas
backend/routes/ -> REST endpoints
backend/controllers/ -> authentication controller
project_documentation.docx -> final project report
Abstract.docx -> formatting and chapter instruction template

Fig 5.1: Development Environment Layout

5.4 Technology Stack Usage Matrix

Technology	Layer	Where It Is Used
React.js 19	Frontend UI	Used in App.jsx, Dashboard.jsx, Login.jsx, StudentPortal.jsx, and all management panels to build reusable component-based screens.
Vite 8	Frontend build tool	Used for fast development server, bundled production build, and modern module handling in the frontend folder.
React Router DOM 7	Frontend routing	Used in App.jsx and ProtectedRoute.jsx to control /, /dashboard, and /profile navigation.
Axios	HTTP client	Used through src/services/api.js; attaches JWT bearer token from localStorage to every protected API request.

CSS	Frontend styling	Used in dashboard.css, App.css, and index.css to style dashboards, panels, grids, tables, buttons, schedule boards, and student portal layouts.
Node.js	Backend runtime	Runs server.js and all CommonJS backend modules.
Express.js 5	Backend framework	Defines REST route groups for auth, users, attendance, notices, courses, subjects, timetable, payments, results, messages, and admin stats.
MongoDB	Database	Stores campus data collections including users, profiles, courses, subjects, timetables, attendance, payments, results, notices, and messages.
Mongoose	ODM	Defines schemas, indexes, validation, relationships, populate behavior, and database operations for all backend models.
JWT	Authentication token	Signed during login and verified by authMiddleware for protected routes.
bcryptjs	Password security	Hashes admin-created passwords and compares login passwords without storing plain text.
Nodemailer	Email service	Sends login ID recovery and OTP password reset emails when SMTP variables are configured.
dotenv	Environment configuration	Loads PORT, MONGO_URI, JWT_SECRET, CLIENT_URL, and SMTP configuration in backend/server.js and email

		service.
cors	Cross-origin access	Allows the Vite frontend at localhost:5173 to call the Express API at localhost:5000.

Presentation Layer: React.js, React Router, CSS, Vite Integration Layer: Axios service with Authorization interceptor Application Layer: Node.js and Express route groups Security Layer: JWT authentication, bcryptjs hashing, role checks Data Layer: MongoDB collections through Mongoose models Email Layer: Nodemailer SMTP for login ID and OTP recovery Configuration Layer: dotenv environment variables and cors frontend origin

Fig 5.2: Technology Stack Layer Diagram

5.5 Frontend Technology Features

React is used to divide the user interface into reusable components. This is important because the admin dashboard contains many independent modules. Each module can manage its own state, form inputs, API calls, and visual layout. For example, `StudentsPanel` handles student form state and filtering; `TimetablePanel` handles course, semester, selected slot, and editor state; `AttendanceForm` handles selected date, lecture, students, and attendance statuses.

Vite is used because it provides a fast development environment for React. It supports modern JavaScript modules and quick reload during development. React Router DOM is used to define the main pages and protect them through `ProtectedRoute`. Axios is used because repeated API calls are required, and the centralized API service automatically attaches the JWT token to protected requests.

5.6 Backend Technology Features

Node.js is used as the backend runtime because it supports JavaScript on the server and works well with Express. Express provides route groups, middleware support, JSON parsing, and structured REST endpoints. The backend uses CommonJS modules and separates code into routes, models, controllers, services, and middleware.

Mongoose is used to model MongoDB data. It allows schema definitions, required fields, default values, enums, indexes, references, timestamps, and populate operations. For example, `Attendance` defines a unique index to prevent duplicate lecture attendance for the same student, while `Subject` defines a unique index for course, semester, and subject code.

5.7 Security and Environment Requirements

The backend requires a secure JWT secret, MongoDB connection URI, frontend client URL, and SMTP settings when email delivery is needed. These values are loaded through `dotenv` instead of being hardcoded. In a production environment, environment variables should be stored securely and never committed to public repositories.

The project also requires CORS configuration so that the frontend application can call backend APIs. In development, the frontend normally runs on `localhost:5173` and the backend runs on `localhost:5000`. The backend CORS configuration allows that origin and credentials.

CHAPTER 6

IMPLEMENTATION AND TESTING

6.1 Implementation Approach

The project was implemented in an iterative manner. The first stage established the frontend and backend folder structure. The second stage added Express server setup, MongoDB connection, and route mounting. The third stage created database models for users, profiles, courses, subjects, timetable, attendance, payments, results, notices, messages, and class assignments. The fourth stage implemented authentication and role-based access. The fifth stage developed admin panels. The sixth stage added faculty attendance and student portal features. The final stage improved documentation, styling, and testing.

```
Project setup -> Backend server -> Database models -> Authentication
Authentication -> Admin user management -> Courses and subjects
Courses and subjects -> Class assignments -> Timetable builder
Timetable builder -> Faculty attendance validation -> Student attendance summary
Core academics -> Payments -> Results -> Notices -> Messages -> Profile
All modules -> Dashboard statistics -> Testing -> Documentation
```

Fig 6.1: Implementation Flow by Module

Module	Implementation Details	Files / Stack Used
Authentication	Login, disabled registration, forgot ID, password reset OTP, JWT token issue, password hashing	authController.js, authRoutes.js, authMiddleware.js, emailService.js, Login.jsx
Admin Dashboard	Statistics cards, notice management, course/subject/student/faculty/timetable/payment/result/message panels	Dashboard.jsx, adminRoutes.js, related components
Student Management	Create/edit student, roll number login ID, filters by course, semester/year, enrollment	StudentsPanel.jsx, userRoutes.js, User.js, Profile.js
Faculty Management	Create faculty account, faculty number login ID, qualification and	FacultiesPanel.jsx, userRoutes.js,

	experience profile	User.js, Profile.js
Course Management	Create/delete course, duration type, department, subject/student counts	CoursesPanel.jsx, courseRoutes.js, Course.js
Subject Management	Course and semester linked subjects, subject code, core/elective type, marks	CourseSubjectsPanel.jsx, subjectRoutes.js, Subject.js
Class Assignment	Class batch, roster, faculty assignment, class teacher, assigned subjects	ClassAssignmentPanel.jsx, classAssignmentRoutes.js, ClassAssignment.js
Timetable	Visual grid, slot editor, subject/faculty/room assignment, conflict detection	TimetablePanel.jsx, timetableRoutes.js, Timetable.js
Attendance	Faculty lecture list, session loading, present/absent records, student summary	AttendanceForm.jsx, AttendanceView.jsx, attendanceRoutes.js, Attendance.js
Student Portal	Home, timetable, attendance, fees, results, messages, profile	StudentPortal.jsx with attendance/payment/result/timetable/message/profile APIs
Payments	Fee structures, manual fee records, assignment, paid status, receipt	PaymentsPanel.jsx, paymentRoutes.js, FeeStructure.js, Payment.js
Results	Result entry, edit result, summary, SGPA/CGPA-style view	ResultsPanel.jsx, resultRoutes.js, Result.js
Notices	Admin notice creation, editing, deletion, role-targeted listing	NoticeForm.jsx, Dashboard.jsx, noticeRoutes.js, Notice.js
Messages	Threaded messages, replies, unread read state, admin visibility	MessagesPanel.jsx, messageRoutes.js, Message.js
Profiles	Self and admin profile updates for user information	ProfilePanel.jsx, profileRoutes.js, Profile.js

6.2 Authentication Module

The authentication module begins at Login.jsx. The login screen uses one identifier field because the system supports different login identifiers for different roles. Students enter roll number, faculty members enter faculty number, and administrators can enter email. The frontend sends the identifier and password to POST /api/auth/login.

On the backend, authController uses a helper to find a user either by institutionalId or by normalized email. If the user is missing, it returns a user-not-found response. If the password does not match, it returns a wrong-password response. If the account is not active, it returns an account inactive response. If all checks pass, it signs a JWT using JWT_SECRET and returns the token and safe user data.

Forgot ID is implemented with registered email. The backend finds the user by email and sends the login ID, which may be institutionalId or email. Forgot password uses an OTP. The OTP is not stored directly; its SHA-256 hash is stored with an expiry time. Reset password checks the OTP, expiry, and new password, then hashes the new password with bcryptjs.

6.3 Admin Management Module

The admin management module is the largest role-specific area. It appears in Dashboard.jsx when the logged-in user has admin role. The dashboard first loads notices and admin statistics. The stats endpoint counts verified students, verified faculty, subjects, courses, notices, attendance entries, payments, paid payments, timetable entries, and result records. These values appear as cards on the admin dashboard.

The admin can create students and faculty. Student creation requires roll number, name, email, and password. Faculty creation requires faculty number, name, email, and password. The backend stores roll number or faculty number in institutionalId, which means the same login logic can identify both types of users. User records store authentication data and Profile records store academic or personal profile data.

Admin user management also includes editing student records. StudentsPanel allows the admin to open an edit form, update student identity/profile fields, and optionally set a new password. Filtering by course, semester/year, and enrollment number helps locate a student from a larger list.

6.4 Academic Management Module

Academic management is implemented through courses, subjects, class assignments, timetable, attendance, and results. Course management defines the program structure. Each course has a course code, course name, semester/year type, total duration, and department. Subject management depends on the course and selected semester/year so the academic structure is not stored as unstructured text.

Class assignment connects a class name and batch with course, semester, department, class teacher, and subjects. It also manages the student roster and faculty subject assignments. This is important because attendance and timetable need to know which students belong to a class batch and which faculty members are connected with which subjects.

Result management allows marks to be recorded for a student by subject, course, semester, maximum marks, grade, and remarks. The result summary route groups entries by semester and computes percentage and grade-point style summaries. The frontend presents these summaries as marksheet-style cards.

6.5 Timetable Module

The timetable module uses a visual schedule grid. The frontend defines days from Monday to Friday and five lecture slots from 08:30 to 12:35. The admin selects a course and semester, chooses a grid slot, selects subject, faculty, and room, then saves the slot through `/api/timetable/slot`.

The backend validates that the faculty exists and is verified. It resolves the course and semester scope and checks for conflicts. A conflict occurs when another entry on the same day overlaps in time and uses the same faculty, the same class/course scope, or the same room. This prevents basic timetable mistakes.

Faculty and students receive read-only timetable views. Faculty timetable queries are filtered to the logged-in faculty ID. Student timetable queries use the student's profile `assignedClass` and `assignedBatch` so the student sees only their schedule.

6.6 Attendance Module

Attendance is linked to timetable rather than being a free-form list. The faculty selects a date. The frontend asks `/api/attendance/faculty/lectures` for the logged-in faculty's lectures on the day of that date. The response includes class options and lecture options. After a lecture is selected, the frontend asks `/api/attendance/faculty/session` to load the lecture, students, and existing attendance statuses.

The backend checks whether the logged-in faculty is assigned to that class, batch, day, start time, and end time. If the faculty is not assigned, the backend rejects the request. This is a critical rule because it prevents faculty from marking attendance for lectures they do not own.

When the faculty saves attendance, the backend updates existing attendance records or creates new records. The Attendance schema has a unique index on student, class, batch, subject, lecture date, start time, and end time. This helps prevent duplicate attendance records for the same lecture.

6.7 Student Portal Module

The student portal is implemented separately from the admin and faculty dashboard. When `Dashboard.jsx` detects a student role, it returns `StudentPortal` instead of the general dashboard layout. `StudentPortal` loads attendance records, attendance summary, payments, results, timetable, messages, and profile in one combined request group.

The home section shows attendance percentage, fee due status, class schedule, attendance subject list, result entries, recent messages, and profile snapshot. Separate sidebar sections allow the student to open My Classes, My Attendance, Fee Details, Results, Messages, and Profile. This makes the student role practical because the student can find information quickly without admin-level controls.

6.8 Fees, Results, Notices, and Messages

The payment module supports both fee structures and manual fee records. An admin can define a fee structure for a class and semester, assign it to students, create manual payment entries, mark payments as paid, and view receipts. Students can view their own payment records and mark applicable payments as paid in the development implementation.

The results module supports marks entry and summary. Admins and faculty can create or edit results. Students can view their own results. The summary groups records by semester and calculates percentage and grade-point style values. This gives the system a marksheet-like output instead of only a raw list of marks.

The notice module lets the admin create notices targeted to student, faculty, or all users. The message module supports threaded communication between students and faculty. Admins can view conversations for oversight, while students and faculty can start conversations, reply in threads, and mark messages read.

6.9 Testing

Testing was planned around module behavior, role restrictions, data validation, and user interface flow. Because the project is a full-stack website, testing must cover frontend actions, backend route responses, database changes, and security restrictions. The following table documents suitable tests for the implemented modules.

Start server and frontend
Check /api/health
Login with admin, faculty, and student identifiers
Verify JWT token is attached to protected requests
Create course -> subject -> class assignment -> timetable
Login as faculty -> load assigned lecture -> save attendance
Login as student -> verify portal data
Create fee/result/notice/message records
Check forbidden actions for incorrect roles
Record pass/fail outcome

Fig 6.2: Testing Workflow

Test Case	Expected Result	Module Tested	Observed/Expected Status
-----------	-----------------	---------------	--------------------------

Login with valid admin email	Admin token and dashboard should load	Authentication, JWT, Dashboard	Pass when backend and database are running.
Login with student roll number	Student portal should open	Authentication and StudentPortal	Pass for admin-created verified student.
Public registration	API should reject with disabled registration message	Auth route	Pass because register returns 410.
Create course	Course appears in course list	CoursesPanel and courseRoutes	Pass when required fields are present.
Create subject for course	Subject appears under selected course and semester	CourseSubjectsPanel and subjectRoutes	Pass with valid course ID.
Create duplicate subject code for same course/semester	API should reject duplicate	Subject unique index	Expected failure with duplicate message.
Save timetable slot	Slot appears in visual timetable	TimetablePanel and timetableRoutes	Pass with valid faculty and subject.
Create conflicting timetable slot	API should reject conflict	Timetable conflict detection	Expected failure.
Faculty loads lectures for date	Only assigned lectures appear	Attendance routes	Pass for faculty with timetable entries.
Faculty marks unassigned lecture	API rejects access	Attendance authorization	Expected failure.
Student views attendance summary	Subject percentages appear	Attendance summary API	Pass after attendance records exist.
Admin creates fee structure	Structure appears and can be assigned	Payment routes	Pass with class and amount.
Student views fee details	Only student-related payments appear	Payment route role filtering	Pass for logged-in student.
Create result entry	Result appears in log and summary	ResultsPanel and resultRoutes	Pass with valid marks.

Create notice for student	Student sees targeted notice	Notice filtering	Pass when role matches.
Student sends message to faculty	Thread appears in message panel	MessagesPanel and messageRoutes	Pass when receiver is valid.

CHAPTER 7

RESULTS AND SCREENSHOTS

7.1 Results

The Smart Campus Management System was successfully implemented as a working MERN stack web application. The project provides secure login, disabled public registration, admin-created student and faculty accounts, dashboard statistics, course management, subject management, class assignment, timetable builder, faculty attendance, student portal, payment management, result management, notices, messages, and profile management.

The result of the project is not limited to a static interface. The frontend sends actual API requests through Axios, the backend validates the requests, MongoDB stores records, JWT protects private endpoints, and Mongoose schemas define the structure of campus data. This demonstrates a practical end-to-end full-stack system.

Screen/Module	Status	Result Explanation
Login Page	Implemented	Supports roll number, faculty number, admin email, forgot ID, and forgot password.
Admin Dashboard	Implemented	Shows statistics and management panels.
Courses	Implemented	Admin can add and delete courses with duration type.
Subjects	Implemented	Admin can add course-semester linked subjects.
Students	Implemented	Admin can create, edit, and filter students.
Faculties	Implemented	Admin can create faculty accounts.
Class Assignment	Implemented	Class batches, rosters, faculty subject assignments, and class teacher are supported.
Timetable	Implemented	Visual timetable builder with

		conflict detection.
Attendance	Implemented	Faculty attendance restricted to assigned lecture slots.
Student Portal	Implemented	Portal sections for classes, attendance, fees, results, messages, profile.
Payments	Implemented	Fee structures, records, paid status, and receipts.
Results	Implemented	Marks entry and summary view.
Notices	Implemented	Role-targeted notices.
Messages	Implemented	Threaded student-faculty communication.
Profile	Implemented	Profile viewing and updating.

7.2 Login Page

```
Screen: Smart Campus Management System login
Fields: Roll number / Faculty number / Admin email, Password
Actions: Login, Forgot ID, Forgot Password
Backend: POST /api/auth/login, /recover-id, /request-password-reset, /reset-
password
Security: bcryptjs comparison, JWT issue, public registration disabled
```

Fig 7.1: Login Page

The login page gives a single entry point for all three roles. The text explains that students use roll number, faculty members use faculty number, and administrators use email. The forgot ID mode accepts registered email and the forgot password mode accepts login ID or registered email, then OTP and new password.

7.3 Admin Dashboard

Screen: Admin Dashboard

Cards: Students, Faculty, Courses, Subjects, Notices, Attendance, Timetable, Fees Paid, Results

Panels: NoticeForm, CoursesPanel, CourseSubjectsPanel, StudentsPanel, FacultiesPanel, TimetablePanel, PaymentsPanel, ResultsPanel, MessagesPanel

Backend: GET /api/admin/stats and module-specific API groups

Fig 7.2: Admin Dashboard

The admin dashboard proves that the system can centralize institutional management. The admin does not need separate pages for every module because the dashboard arranges panels and statistics in one working area.

7.4 Course and Subject Screens

```
Course screen: course code, course name, sem/year type, total sem/year, department
Subject screen: course selector, semester/year selector, subject code, subject
name, subject type, theory marks, practical marks
Backend: /api/courses and /api/subjects
Database: Course and Subject collections
```

Fig 7.3: Course and Subject Management

The course and subject result shows that academic structure is not hardcoded. Admin can define a course and then add subjects for selected course and semester/year combinations.

7.5 Student and Faculty Screens

```
Student screen: Add Student, View Student, course filter, semester/year filter,
enrollment filter, edit action

Faculty screen: Add Faculty, faculty ID, name, email, qualification, experience

Backend: /api/users/students and /api/users/faculty

Database: User and Profile collections
```

Fig 7.4: Student Management Screen

The student and faculty screens show the admin-created account model. Roll number and faculty number become institutional login IDs, which prevents unauthorized public account creation.

7.6 Timetable Builder

```
Screen: Monday-Friday timetable grid
Slots: 08:30-09:15, 09:20-10:05, 10:10-10:55, 11:00-11:45, 11:50-12:35
Editor: Subject, Faculty, Room/Lab
Backend: PUT /api/timetable/slot
Validation: conflict detection for faculty, class/course, and room
```

Fig 7.5: Timetable Builder

The timetable builder gives a graphical way to manage class schedules. It improves usability because the admin can see filled and vacant slots rather than entering schedule records only in forms.

7.7 Faculty Attendance Screen

Screen: Lecture Attendance

Flow: Select date -> Select assigned class -> Select lecture -> Load students -> Mark present/absent -> Save

Backend: /api/attendance/faculty/lectures and /api/attendance/faculty/session

Security: faculty cannot mark unassigned lecture

Fig 7.6: Faculty Attendance Screen

The attendance screen is one of the strongest workflow results. It connects timetable and attendance so that attendance is not arbitrary. Existing attendance can be updated for a lecture, and new attendance records can be created for students in the selected class batch.

7.8 Student Portal

Screen: Student sidebar portal

Sections: Home, My Classes, My Attendance, Fee Details, Results, Messages, Profile

Data: attendance, summary, payments, results, timetable, messages, profile

Frontend: StudentPortal.jsx

Fig 7.7: Student Portal Dashboard

The student portal result gives students a realistic academic dashboard. It shows attendance percentage, fee due status, schedule, result records, messages, and profile snapshot. Separate navigation sections make it easier for students to access details.

7.9 Fee, Result, Notice, and Message Screens

Fees: structures, manual records, assignment, paid status, receipt
Results: marks entry, grade, remarks, semester summary, CGPA snapshot
Notices: role-targeted announcements
Messages: threaded conversations with unread state

Fig 7.8: Fee, Result, Notice, and Message Screens

These supporting modules make the portal more complete. They show that the project is not only a login and attendance system; it also supports financial records, academic results, institutional announcements, and user communication.

7.10 Discussion

The implemented system demonstrates how a campus ERP-like portal can be built with a modular full-stack approach. The separation of roles makes the system practical. Admin users perform institutional management, faculty users perform teaching-related tasks, and student users consume personal academic information. This separation reduces confusion and helps protect data.

A major result is the removal of open registration. This is important because a campus portal must not allow anyone on the internet to create a student or faculty account. The project handles this by making admin account creation the source of verified users. Another major result is attendance restriction through timetable validation. Faculty members cannot mark attendance for random classes; they must be assigned to the lecture slot.

The timetable and subject modules also improve the system. Subjects are linked with courses and semesters, and the timetable builder uses those records. This creates a stronger academic structure than storing subject names as plain text in unrelated fields.

7.11 Advantages and Limitations

Advantages	Limitations
Centralized management of campus data across roles.	MongoDB and backend server must be available for full operation.
Admin-created accounts reduce fake registration risk.	SMTP must be configured for real OTP email delivery.
Role-based dashboards improve usability.	Current project does not include a dedicated mobile app.
Faculty attendance is connected with assigned timetable lectures.	Biometric or QR-based attendance is not implemented yet.
Course and subject structure is organized.	Advanced analytics and reporting dashboards can be expanded.
Student portal gives quick access to academic information.	Production deployment, monitoring, and backups are not fully configured.
JWT and bcryptjs improve application security.	File uploads are represented by URLs and can be enhanced with managed storage.
Modules are separated into reusable frontend	Automated test suite can be added in future versions.

components and backend route groups.	
--------------------------------------	--

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Smart Campus Management System successfully achieves the objective of designing and developing a secure, role-based, full-stack campus portal. It replaces scattered manual processes with a centralized digital system for users, profiles, courses, subjects, class assignments, timetable, attendance, payments, results, notices, messages, and dashboard statistics.

The project demonstrates the use of React for frontend screens, Vite for development and build tooling, React Router for navigation, Axios for API communication, Node.js and Express.js for backend APIs, MongoDB for database storage, Mongoose for schema modeling, JWT for protected sessions, bcryptjs for password hashing, Nodemailer for email recovery, dotenv for configuration, and CORS for frontend-backend communication.

The most important functional achievements are admin-controlled account creation, institutional login identifiers, OTP-based recovery, lecture-linked attendance, visual timetable management, student portal dashboard, fee and result records, role-targeted notices, and threaded messaging. These features make the project realistic for a campus environment and suitable as a major academic project.

8.2 Limitations

- The system currently requires local or configured MongoDB availability and does not include production database replication.
- SMTP settings must be configured for real email delivery; otherwise email recovery works only with development fallback behavior.
- The current implementation is web-based and does not include a packaged Android or iOS mobile application.
- Biometric attendance, QR code attendance, and location-based attendance are not implemented.
- Photo and document upload support is basic and can be improved with cloud storage.

- Advanced analytics dashboards for attendance trends, fee collection, faculty workload, and subject performance are not included yet.
- The project does not include a complete production deployment setup with HTTPS, CI/CD, monitoring, centralized logs, and automated backups.
- Automated unit tests and integration tests can be expanded further.

8.3 Future Enhancement

Future Enhancement	Explanation
Mobile Application	A React Native mobile app can be developed so students and faculty can access timetable, notices, fees, messages, and attendance from smartphones.
Biometric or QR Attendance	Attendance can be enhanced using fingerprint devices, face verification, QR codes, or location-based checks to reduce proxy attendance.
Push Notifications	Firebase Cloud Messaging or similar services can send notices, fee reminders, timetable updates, and result alerts.
Advanced Analytics	Admin dashboards can include charts for attendance trends, fee collection, subject performance, and faculty workload.
Cloud Deployment	The backend, frontend, and database can be deployed on cloud infrastructure with domain, SSL, and monitoring.
Backup and Restore	Scheduled MongoDB backups and restore procedures can protect institutional records.
Document Management	Students and admins can upload ID documents, profile photos, certificates, assignments, and reports using secure file storage.
Parent Portal	Parents can receive restricted access to student attendance, fees, and academic progress.
AI Helpdesk	A chatbot can answer common student queries about timetable, attendance, fees, and academic procedures.
Audit Logs	Every admin action can be logged for accountability and compliance.
Role Granularity	Additional roles such as accountant, examination cell, department head, and class coordinator can be added.
Automated Testing	Unit, integration, and end-to-end tests can be added

	for stable production releases.
--	---------------------------------

8.4 Final Summary

In final summary, the Smart Campus Management System provides a strong foundation for a complete campus portal. It is detailed enough to represent real academic and administrative workflows while remaining understandable as a student major project. The implementation shows how modern web technologies can solve practical campus problems by connecting users, data, processes, and security into one system.

REFERENCES

- [1] React Official Documentation, <https://react.dev>
- [2] Vite Official Documentation, <https://vite.dev>
- [3] React Router Documentation, <https://reactrouter.com>
- [4] Axios Documentation, <https://axios-http.com>
- [5] Node.js Official Documentation, <https://nodejs.org>
- [6] Express.js Documentation, <https://expressjs.com>
- [7] MongoDB Documentation, <https://www.mongodb.com/docs>
- [8] Mongoose Documentation, <https://mongoosejs.com>
- [9] JSON Web Token Documentation, <https://jwt.io>
- [10] bcryptjs Package Documentation, <https://www.npmjs.com/package/bcryptjs>
- [11] Nodemailer Documentation, <https://nodemailer.com>
- [12] MDN Web Docs, <https://developer.mozilla.org>
- [13] npm Documentation, <https://docs.npmjs.com>
- [14] Git Documentation, <https://git-scm.com/doc>
- [15] Visual Studio Code Documentation, <https://code.visualstudio.com/docs>
- [16] MongoDB University and learning resources for schema design and aggregation concepts.
- [17] General academic references on campus ERP, student information systems, and web application architecture.

PLAGIARISM REPORT COPY

This page is reserved for attaching the plagiarism report copy as required by the abstract/template instruction. The final plagiarism report should be generated through the institution-approved plagiarism checking system and attached here before physical submission.

Report Status: To be attached by project team before final submission.

Similarity Percentage: _____

Checked By: _____

Date: _____

JOURNAL PROOF / PUBLISHED PAPERS / PROFILE COPY

This page is reserved for journal proof, published paper evidence, or one-page profile copy as required by the abstract/template instruction. If the project team publishes or submits a paper related to the Smart Campus Management System, the acknowledgement, acceptance proof, publication proof, or author profile copy should be attached in this section.

Paper / Profile Status: To be attached by project team if applicable.

Paper Title: _____

Journal / Conference Name: _____

Publication / Submission Date: _____